

CSCI 235: Software Design & Analysis II

Day 1 — Classes: An Overview

Tong Yi

Hunter College, CUNY

Fall 2026

- 1 Question: A First Class
- 2 Rectangle: A Richer Class
- 3 References

- This course is about **designing** data structures, not just using ones someone else already built.
- Philosophy: *build it yourself first* — you will implement your own classes and data structures before relying on the C++ Standard Library (STL).
- Big-O analysis is not a separate topic; it is discussed alongside every structure we build.
- The course is project-driven: most of the semester's work is a sequence of hands-on programming projects.

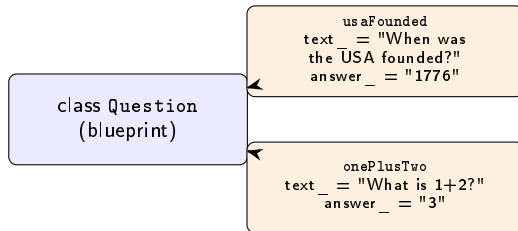
Today's goal

Review how C++ classes encapsulate data and behavior, how constructors create objects, and how references let us work with objects without unnecessary copying.

- 1 Question: A First Class
- 2 Rectangle: A Richer Class
- 3 References

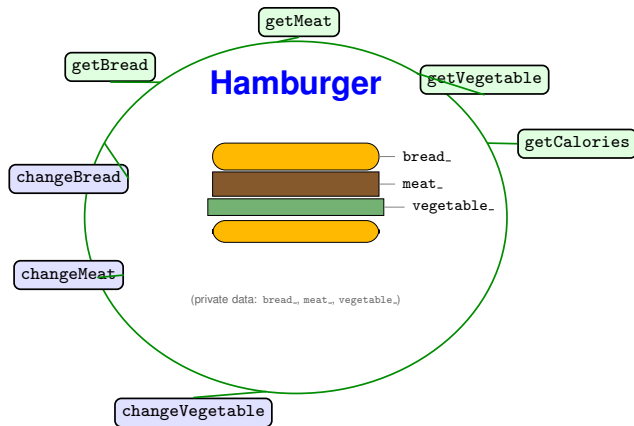
What Is a Class?

- A **class** is a *blueprint*: it describes what data (state) and what behavior (functions) every object of that type will have.
- An **object** is a concrete instance built from that blueprint. Each object gets its own memory.



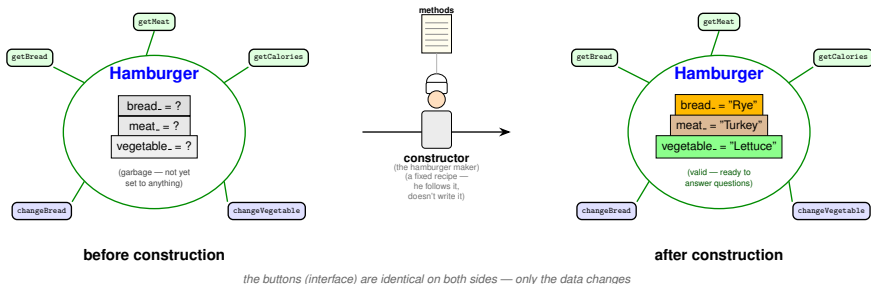
We will use `Question` as our very first class — it is small enough to hold in your head all at once, and you already know what it represents.

Classes Are Everywhere: Hamburger



A Hamburger object hides its `bread_`, `meat_`, and `vegetable_` data inside; the buttons around the rim are its **public interface** — **green** buttons *read* data (accessors), **blue** buttons *change* it (mutators). Every class has this same shape, including Question.

What Does a Constructor Actually Do?

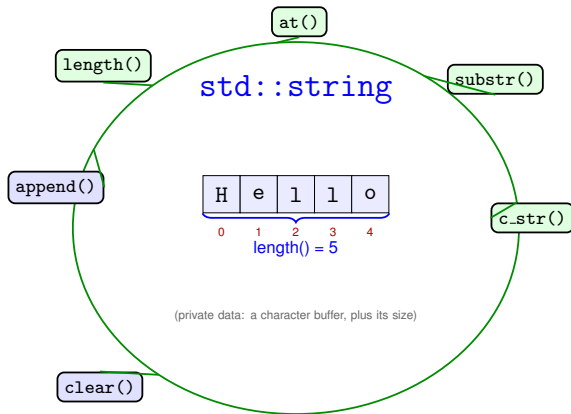


```
Hamburger::Hamburger(const std::string& bread, const std::string& meat,
                     const std::string& vegetable)
: bread_(bread), meat_(meat), vegetable_(vegetable) { }
```

A common misconception, worth heading off

The constructor does **not** create the buttons around the rim — those already exist for every object the moment the class is compiled. It guarantees only that, by the time anyone calls `getMeat()`, there is something **true** to report — not garbage.

You Have Already Used a Class: `std::string`



Even `std::string` — a type you have used since your very first C++ program — is a class: private data (a character buffer, plus its size) exposed only through a public interface like `length()`, `at()`, and `append()`.

The Question Class: Interface

```
//File name: /Users/laptopuser/Documents/courses_macbook_pro/cs235_f26/  
class_day_f26/day1_classOverview/code/question/Question.hpp  
#ifndef QUESTION_HPP  
#define QUESTION_HPP  
#include <string>  
class Question {  
    public:  
        Question();  
        Question(const std::string& text, const std::string& answer);  
        std::string text() const;  
        bool checkAnswer(const std::string& reply) const;  
  
    private:  
        std::string text_;  
        std::string answer_;  
};  
#endif
```

public section = the interface (what other code may call). **private** section = hidden state
— not directly reachable from outside the class.

Member Functions and `const`

```
#include "Question.hpp"

Question::Question() : text_(""), answer_("") {
}

Question::Question(const std::string& text, const std::string& answer) : text_(text), answer_(answer) {
}

std::string Question::text() const {
    return text_;
}

bool Question::checkAnswer(const std::string& reply) const {
    return answer_ == reply;
}
```

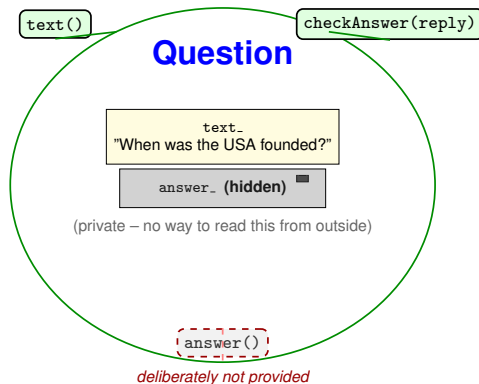
- A **member function** is declared inside the class, then defined outside it as `Question::functionName(...)` — the `::` ties the definition back to its class.
- Trailing `const` (as in `text() const`) is a promise: *this function will not modify the object's data members.*

Design Decision: No answer() Method

Look closely at the interface

Question gives you `text()` and `checkAnswer(reply)`. There is **no** `answer()` that hands back `answer_`.

- This is **intentional**, not an oversight.
- The only way to interact with the correct answer is to *guess* it and ask `checkAnswer` whether you were right — exactly like taking a real quiz.
- Because `answer_` is private, nothing outside the class — not even the quiz-running code — can read it directly.
- This is **encapsulation**: bundling data with the functions that control access to it.



Passing Strings by `const std::string&`

```
Question(const std::string& text, const std::string& answer);
```

- The constructor parameters are `const std::string&`, not plain `std::string`.
- `&` means **no copy** of the caller's string is made just to pass it in; `const` promises the constructor will not modify it.
- We will name this pattern precisely in a few minutes — for now, just notice it: **whenever a function only needs to read a string, it takes it by `const&`.**

Passing an Object by `const` Reference

```
Question usaFounded("When was the USA founded?", "1776");  
testQuestion(usaFounded);
```

```
void testQuestion(const Question& ques) {  
    std::cout << ques.text() << '\n';  
    ...  
    std::cout << std::boolalpha << ques.checkAnswer(reply);  
}
```

- Same idea as `const std::string&`, now applied to a whole **Question object**: `testQuestion` reads `ques` but never copies or modifies it.
- Keep this pattern in mind — Section 3 formalizes exactly what is happening here.

- 1 Question: A First Class
- 2 Rectangle: A Richer Class
- 3 References

Let's Look at a Richer Class

- Question was deliberately minimal — one job, two short member functions.
- Rectangle will let us explore more of what a C++ class can do:
 - **overloaded** constructors (more than one way to build an object)
 - constructors that **validate** their input
 - **copying** an object, and **assigning** one object's value to another

The Rectangle Class: Interface

```
#ifndef RECTANGLE_HPP // do not write ifndef as ifdef, otherwise, Rectangle class is not recognized
#define RECTANGLE_HPP
class Rectangle {
public:
    Rectangle(); // set length and width to be 2 and 1 respectively
    Rectangle(double length, double width);
    Rectangle(const Rectangle& rect);
    // overload the assignment operator
    Rectangle& operator=(const Rectangle& rect);
    double length() const;
    double width() const;
    double area() const;
    double perimeter() const;
    void setLength(double length);
    void setWidth(double width);
    void print() const;

private:
    double length_;
    double width_;
};
#endif
```

public section = the interface. **private** section = hidden state (*encapsulation*, same idea as Question).

Interface vs. Implementation: .hpp and .cpp

- `Rectangle.hpp` declares **what** the class can do — function signatures only, no bodies.
- `Rectangle.cpp` defines **how** each function actually works. Every definition is written `ReturnType Rectangle::functionName(...) { ...}` — `::` is the *scope resolution operator*, tying the definition back to its class.

Why bother separating them?

Anyone can read `Rectangle.hpp` to learn how to *use* `Rectangle` without reading a line of your implementation — and you can hand someone a compiled `.o` file plus the header, with no source code included at all.

- A **constructor** is a special member function that runs automatically when an object is created — same name as the class, **no return type**.
- Rectangle **overloads** its constructor — several versions, same name, different parameter lists:
 - Rectangle() — the **default** constructor
 - Rectangle(double length, double width) — a **parameterized** constructor
 - Rectangle(const Rectangle& rect) — the **copy** constructor (its own special case — more shortly)

The Default Constructor

```
Rectangle::Rectangle() : length_(2), width_(1) {  
}
```

- The part after the colon is a **member initializer list**: it sets `length_` and `width_` *before* the constructor body runs — the preferred way to initialize members in C++.
- Our default constructor **guarantees** that `Rectangle r1;` starts as a valid 2×1 rectangle. That guarantee comes from *this specific constructor* — it is not a promise C++ makes about objects in general.

The Parameterized Constructor

```
Rectangle::Rectangle(double length, double width)
: length_(length > 0 ? length : 2),
  width_(width > 0 ? width : 1) {
}
```

- The ternary `length > 0 ? length : 2` **guards** against bad input: an invalid length silently falls back to the safe default 2 (likewise 1 for width).
- This enforces a **class invariant**: a Rectangle's `length_` and `width_` are *always* positive.

Getters and Setters Protect the Invariant

```
void Rectangle::setLength(double length) {  
    if (length > 0) {  
        length_ = length;  
    }  
}
```

- `length()` / `width()` — **getters**: read-only access to the private data.
- `setLength` / `setWidth` — **setters**: the *only* way outside code can change `length_`/`width_` after construction.
- `setLength` silently **rejects** an invalid value, while the constructor **substitutes a safe default**. Both protect the same invariant: `length_` and `width_` remain positive — that is only possible because they are private.

The Copy Constructor

```
Rectangle::Rectangle(const Rectangle& rect)
    : length_(rect.length_), width_(rect.width_) {
}
```

- Builds a **brand-new** object as a copy of an existing one.
- Its parameter, `const Rectangle& rect`, passes the source object **by reference** — we will see exactly why in Section 3.
- Runs automatically whenever a new object is initialized from an existing object:
`Rectangle r3(r2);` or `Rectangle r4 = r3;`

Copy Assignment: operator=

```
Rectangle& Rectangle::operator=(const Rectangle& rect) {  
    length_ = rect.length_;  
    width_  = rect.width_;  
    return *this;  
}
```

- operator= **overloads** the = symbol so it works on Rectangle objects.
- Copies the data from an **existing** object into another **existing** object (contrast with the copy constructor, which *builds* a new one).
- Returns Rectangle& (not void) so assignments can chain: a = b = c;

Copy Constructor vs. Copy Assignment

```
Rectangle r4 = r3;    // copy constructor  
  
Rectangle r5;  
r5 = r3;              // assignment operator
```

Rule of thumb

Type `x = y`; *looks* like assignment but is **initialization** — the copy constructor runs. `operator=` only runs when the left-hand object **already exists**.

Let's Trace RectangleTest.cpp Together

```
//File name: /Users/laptopuser/Documents/courses_macbook_pro/cs235_f26/class_day_f26/
//day1_classOverview/code/RectangleTest.cpp
#include <iostream>
#include "Rectangle.hpp"

int main() {
    // Instantiate Rectangle object r1 using the default constructor.
    Rectangle r1;
    r1.print();

    // Instantiate rectangle r2 with length 3 and width 1.
    Rectangle r2(3, 1);
    r2.print();

    // Attempt to change the length of r2 to -1. Then change its width to 2.
    r2.setLength(-1); // invalid operation
    r2.setWidth(2);
    r2.print();

    Rectangle r3(r2); // direct-initialization
    Rectangle r4 = r3; // copy-initialization

    Rectangle r5;      // default constructor
    r5 = r3;           // overloaded assignment operator

    return 0;
}
```

We'll compile and run this program live and watch each object's state change — keep an eye on r2 right after setLength(-1), and on r3/r4/r5 as we compare copying and assignment.

- 1 Question: A First Class
- 2 Rectangle: A Richer Class
- 3 References

What Is a Reference?

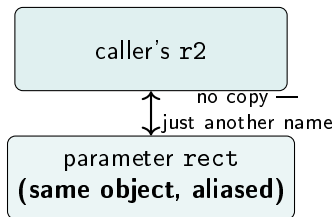
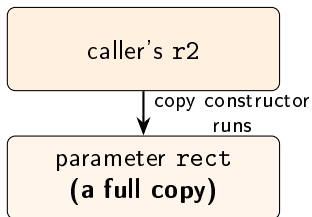
- A **reference** is an *alias* — another name for a variable that already exists. It is **not** a copy, and it is not a new object.
- Syntax: `Type& name = existingVariable;`

```
int x = 5;  
int& ref = x;    // ref is just another name for x  
ref = 10;        // this changes x too!
```

We already used this twice today

- `Question(const std::string& text, ...)` — avoids copying two strings every time a `Question` is built.
- `testQuestion(const Question& ques)` — avoids copying an entire `Question` object on every call.

Pass-by-Value vs. Pass-by-Reference



pass-by-value: `void f(Rectangle rect)`

pass-by-reference: `void f(const Rectangle& rect)`

- Passing an existing `Rectangle` **by value** creates a **copy**; for a class holding much more data than two doubles, that cost can add up.
- **Pass-by-value:** modifying `rect` can't affect the caller's object — `rect` is a copy.
- **Pass-by-const-reference:** no copy is made; `const` is what prevents modifying the caller's object.

For now, a good default

Pass larger objects you only need to read by `const` reference.

- A **class** (Question, Rectangle) is a blueprint; **encapsulation** — private data plus a controlled public interface — lets a class guarantee its own invariants. Question's missing answer() and Rectangle's validated constructors/setters are the same idea.
- **Overloaded constructors, initializer lists, the copy constructor, and operator=** all revolve around building and copying objects correctly.
- A **reference** is an alias, not a copy. For now, a good default: pass larger objects you only need to read by `const` reference.

Next time: what if one class is a *specialized version* of another?
MultipleChoice is a kind of Question; Square is a kind of Rectangle.